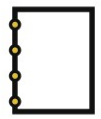
Machine learning: the workflow, from loans data to a first model

What does it mean for a computer to learn from examples, and how do we know it did?

AI and Data Science Professional | Session 14 of 24 | Module D: Machine learning
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Name the parts of a supervised learning problem: records, features, target, training set and test set



Prepare data for a model: impossible values, imputation with a missing-indicator and a new feature, and see why a baseline model comes first



Split the data, fit a first model with scikit-learn, and explain the accuracy paradox on an imbalanced target

Your two hours: concepts first, then a Python machine learning lab

0:00	10	Recall and the question of the day
0:10	45	Concepts: picture, plain words, one example by hand
0:55	10	Break
1:05	45	Lab: predict, run, read, change
1:50	10	Exit check, keep and next

Ideas with pictures first. Then you open the notebook, prepare the loans and fit a first model.

Three questions from last time

1. In the regression on weekly orders, what did a footfall coefficient of 0.0188 mean?

2. What is a residual?

3. The promotion coefficient was 6.8. Does that prove the promotion caused the extra orders?



Answer aloud first. Then we check together.

Answers to the three questions from Session 13

	Question	Short answer
	**In the regression on weekly orders, what did a footfall coefficient of 0.0188 mean?**	About 18.8 more orders for each 1,000 more people at the door, holding promotion and rainfall fixed.
	**What is a residual?**	The actual value minus the predicted value for one record. A residual plot checks whether the errors have an even spread.
	**The promotion coefficient was 6.8. Does that prove the promotion caused the extra orders?**	No. It shows an association. A cause needs an experiment or strong outside evidence.

Words we use today

	Word	What it means
	**Supervised learning**	A model learns the link between features (inputs) and a known target (the answer) from past records, then predicts the target for new records.
	**Features and target**	Features are the input variables. The target is the variable we predict. Statistics says independent and dependent variable.
	**Training set and test set**	The training set is what the model learns from. The test set is held back and used once at the end to score the model.
	**Baseline model**	The simplest possible model, such as always predicting the majority class (DummyClassifier). Every real model has to beat it.
	**Class imbalance (accuracy paradox)**	One class is far more common than the other, so a model that ignores the rare class can still have high accuracy.

A rule you write in Excel and a rule a model learns from examples

Excel IF rule

=IF(instalment_to_income>0.6,"Risky","OK")

The 0.6 is an illustrative hunch.

Flags 1,623 of 8,000 loans.

Supervised learning

Show 8,000 past loans and what happened.

The computer finds the rule.

Test it on unseen loans.

Excel: you give the rule. Machine learning: you give examples.

Supervised learning: features go in, the target comes out

FEATURE	FEATURE	FEATURE	FEATURE	TARGET
age	monthly_income_ugx	loan_amount_ugx	previous_late_payments	default
48	505,300	1,583,200	1	0
35	692,600	3,366,600	1	0
40	411,800	5,081,300	0	1
26	440,000	1,763,700	1	0

Each row is one record (an observation). The model reads the features and predicts the target.

Four real records of loans.csv. default is 1 for a borrower who stopped repaying.

The Excel word, the statistics word and the machine learning word for each part

	In Excel	In statistics	In machine learning
	**A row**	Observation	Instance (record)
	**The input columns**	Independent variables (predictors)	Features
	**The column you want to fill in**	Dependent variable (response)	Target (label, for a class)
	**A yes or no column**	Binary variable	Binary classification target
	**Rows kept out of the analysis**	Hold-out data	Test set

This course says feature and target from here on, and statistics words when the topic is explanation.

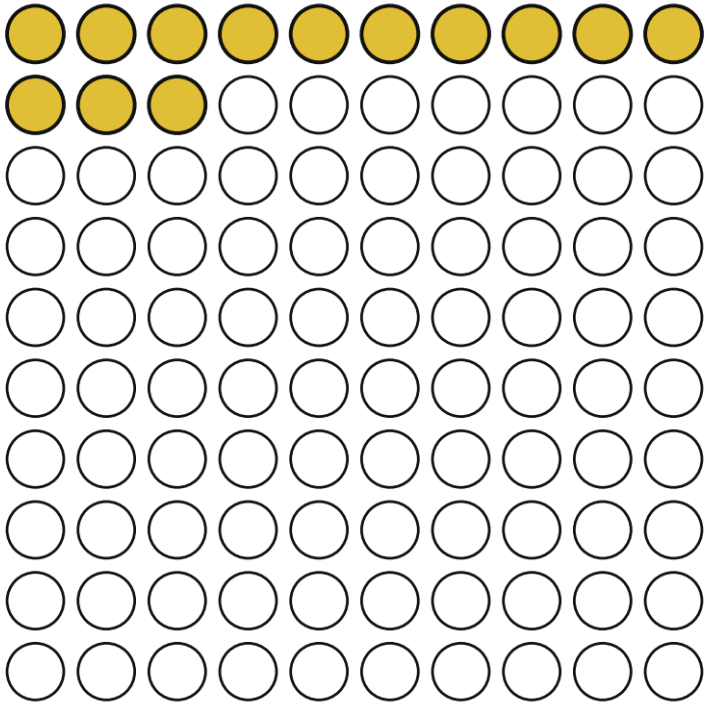
Two kinds of supervised problem: regression predicts a number, classification predicts a category

	Problem type	The target is	Example from our data	Where
	**Regression**	A number	Weekly orders of a Lakeside branch	Session 13
	**Binary classification**	One of two classes (positive class and negative class)	Default: 1 (positive) or 0 (negative)	Today and Session 15
	**Multiclass classification**	One of three or more classes	The theme of a customer message (queue, price, staff and more)	Session 21

About 1 loan in 8 defaults: this is class imbalance

12.55%

1,004 of the 8,000 loans defaulted (the positive class). 6,996 were repaid (the negative class). One class is far more common: class imbalance.



About 13 in every 100 loans default

A baseline model that predicts repaid for every loan is 87.45 percent accurate

Accuracy

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}$$

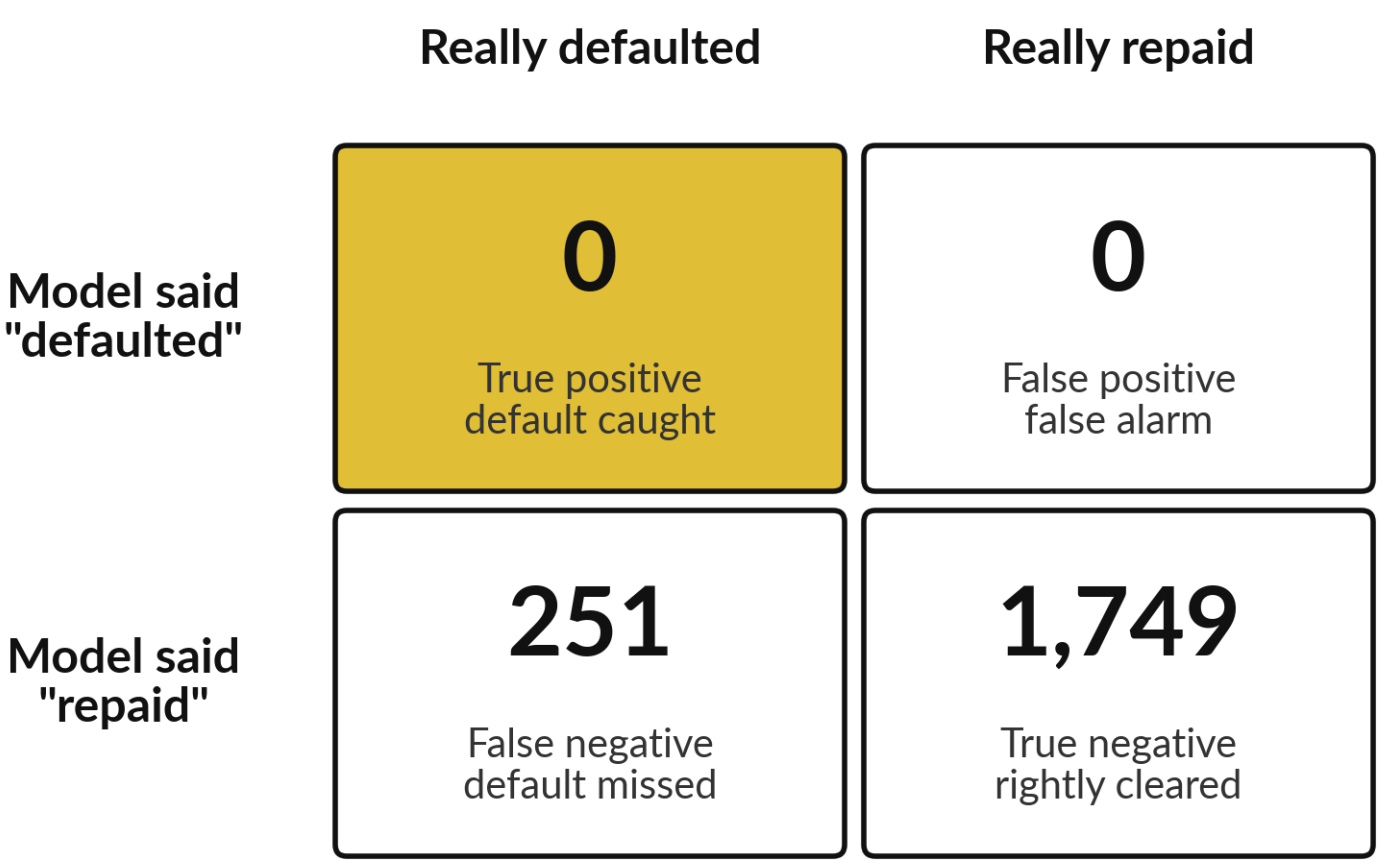
Baseline on our test set

$$\frac{1{,}749}{2{,}000} = 0.8745$$

Symbol	What it means	In our data
correct predictions	Predictions that equal the true answer.	1,749 repaid loans
all predictions	Loans in the test set.	2,000
baseline model	Predicts the majority class every time (DummyClassifier, strategy most_frequent).	always repaid

Every other model has to beat 0.8745. Accuracy is the share of correct predictions.

The accuracy paradox: 87.45 percent accurate and not one default caught



Accuracy hides the rare class.

Recall (sensitivity): the share of real defaults caught. Here 0 of 251.

A bank needs the defaults.

This is the accuracy paradox.

Look at the data before you model it: three problems in the loans file

	Problem	What we found	What we do
	**Impossible values**	2 ages that cannot be right: 0 and 199	Replace with the median of the valid ages, 36
	**Missing values**	379 blank incomes (4.7 percent)	Median imputation, plus a missing-indicator flag
	**A column that leaks the answer**	collection_calls is above 0 for 100 percent of defaulted loans	Leave it out of the features

Every change goes in a data cleaning log with its counts, as in Session 8.

Imputation with a missing-indicator: flag the blank first, then fill it with the median

Step 1: make the flag. Step 2: fill the blank with the median.

loan_id	monthly_income_ugx (raw)	income_missing	monthly_income_ugx (imputed)
N00001	505,300	0	505,300
N00006	blank	1	642,600
N00002	692,600	0	692,600
N00012	blank	1	642,600

Median of the 7,621 known incomes: 642,600 UGX. Make the flag first: after filling, the blanks are gone.

Blank incomes default at 12.1 percent, the others at 12.6. The flag keeps that information.

Feature engineering: build a new feature, the instalment-to-income ratio, from three existing variables

Ratio

$$\text{instalment_to_income} = \frac{1.15 \times \text{loan_amount}}{\text{term_months} \times \text{monthly_income}}$$

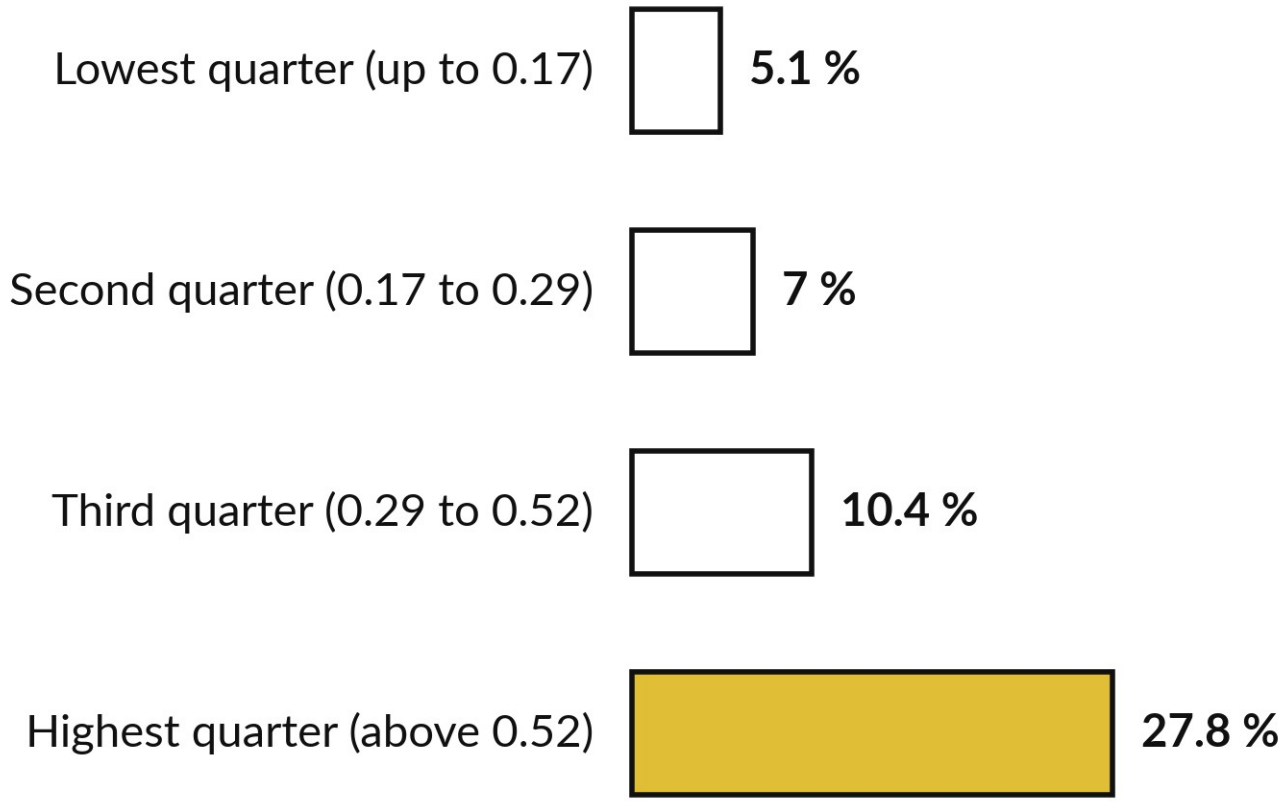
Loan N00001

$$\frac{1.15 \times 1{,}583{,}200}{6 \times 505{,}300} = 0.60$$

Symbol	What it means	In our data
loan_amount, term_months	Loan size in UGX and the number of months to repay it.	1,583,200 UGX, 6 months
monthly_income	Income per month in UGX, after imputation.	505,300 UGX
1.15	Illustrative assumption: the borrower repays the loan plus 15 percent in charges.	not in the file

Share of monthly income for one instalment (1.15 is assumed). Median 0.29, highest 9.1.

Default rate climbs with the instalment-to-income ratio: 27.8 percent in the highest quarter



Each bar is the percentage of loans in that group that defaulted.

Four equal groups of 2,000 loans, cut by the ratio.

The highest group defaults at 5.4 times the rate of the lowest.

Use only what is known when the decision is made: data leakage

Known at approval

age, income, loan amount, term

savings, previous late payments

instalment-to-income ratio

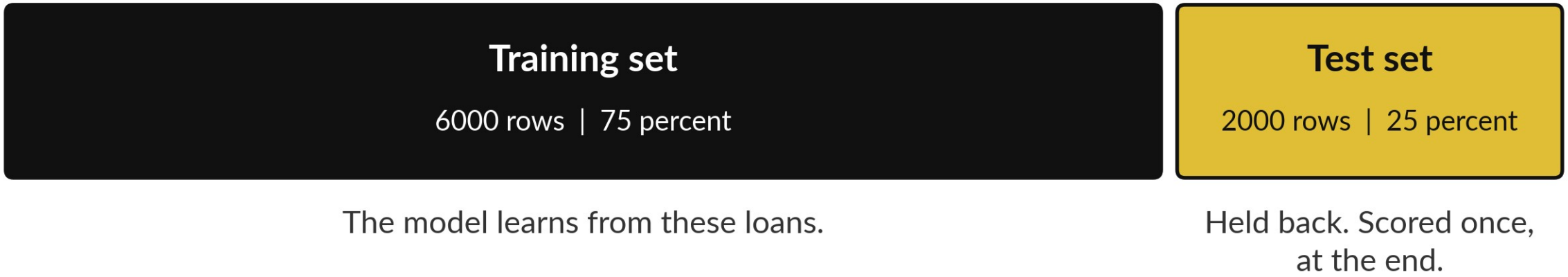
Recorded after the loan goes bad

collection_calls is above 0 for 100 percent of defaulted loans and 21 percent of repaid ones.

It looks brilliant and is useless at approval.

Leave it out. Session 16 studies data leakage.

Hold back a test set before the model sees anything: the train and test split



The function train_test_split shuffles the 8,000 loans and cuts them in two.

Stratifying keeps the default rate the same in both parts

Part	Loans	Defaults	Default rate
All loans	8,000	1,004	12.55 %
Training set	6,000	753	12.55 %
Test set	2,000	251	12.55 %

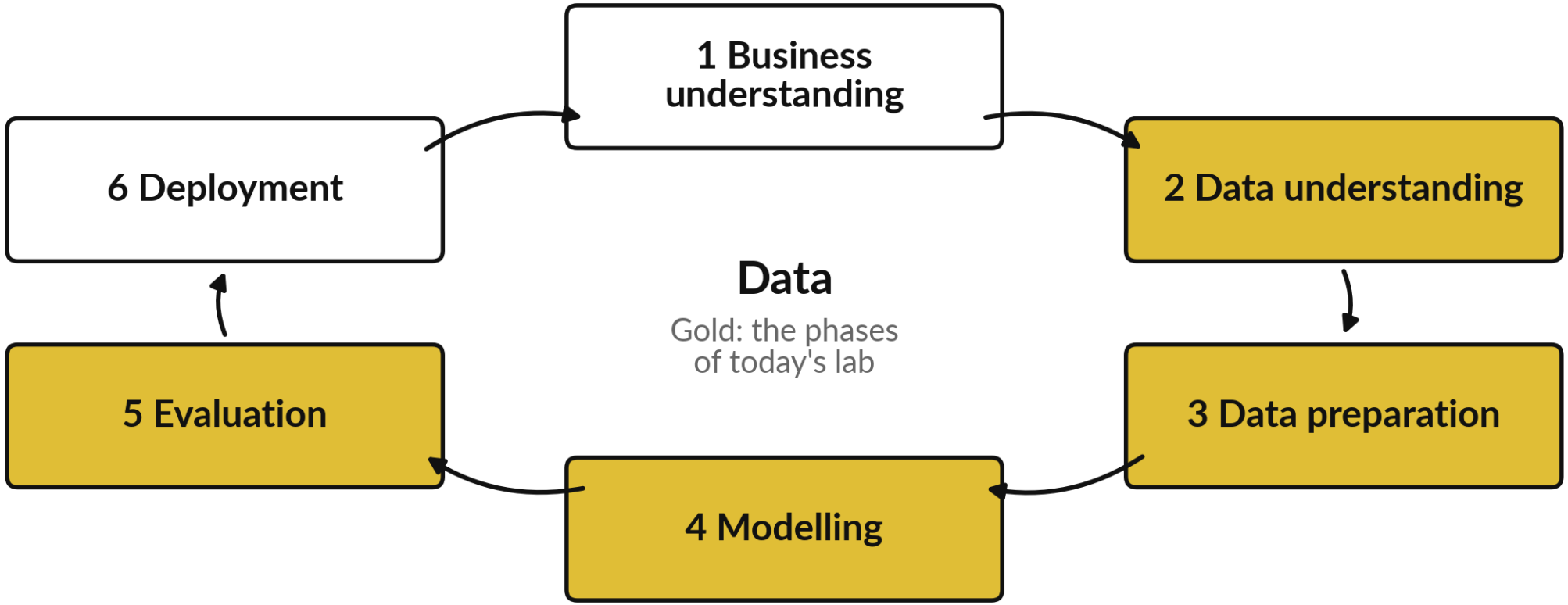
stratify=y keeps the class proportions in both parts. random_state=42 fixes the shuffle, so you and I get the same split.

The train, validate, test discipline: tune on the validation set, use the test set once



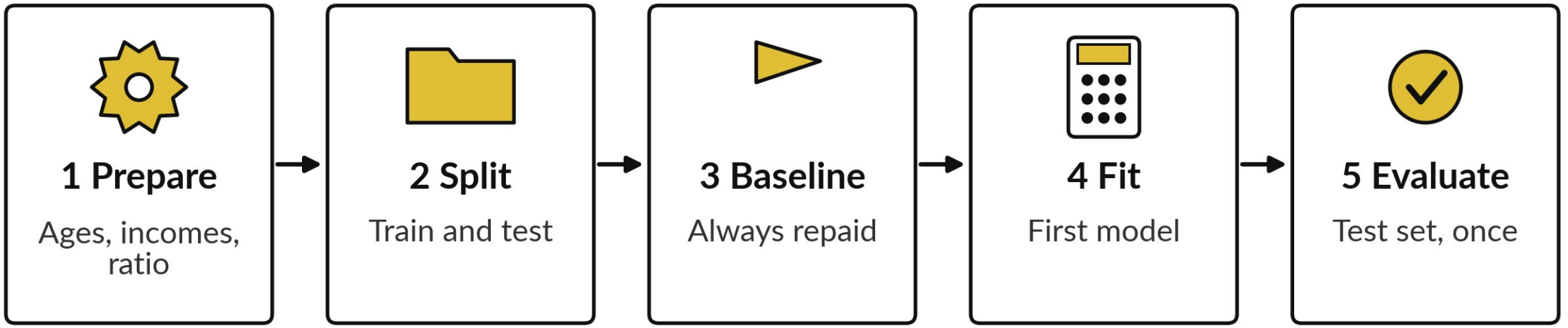
Illustrative sizes. Today: two parts. Session 17: cross-validation plays the validation set.

CRISP-DM is the standard six-phase cycle for a data and machine learning project



CRISP-DM stands for Cross-Industry Standard Process for Data Mining. The arrows loop.

Our workflow in five steps, from the raw loans to a score on the test set



Preparation comes first and the baseline comes before the model. The test set is used only in step 5.

The safe order: split first, then let a pipeline learn the median from the training rows only

```
X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.25, stratify=y,
    random_state=42)
imputer = SimpleImputer(strategy='median')
tree = DecisionTreeClassifier(
    max_depth=3, random_state=42)
pipe = make_pipeline(imputer, tree)
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

In plain words

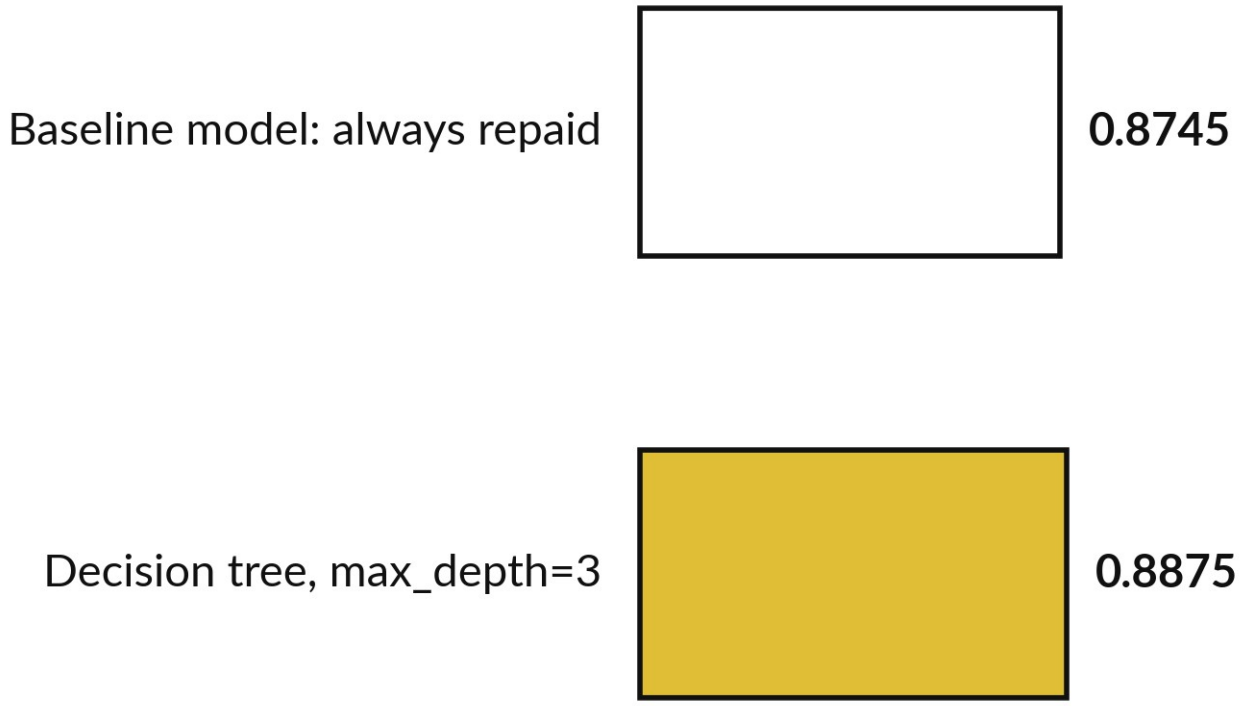
Lines 1 to 3: split first. X_raw still has its blanks.

Lines 4 to 7: an imputer, a tree, and a pipeline that chains them.

Line 8: fit learns the median from the training rows only: UGX 643,200, against 642,600 on all loans.

Line 9: test loans get the training median.

Fit and predict: the first model beats the baseline model by only 1.3 points of accuracy



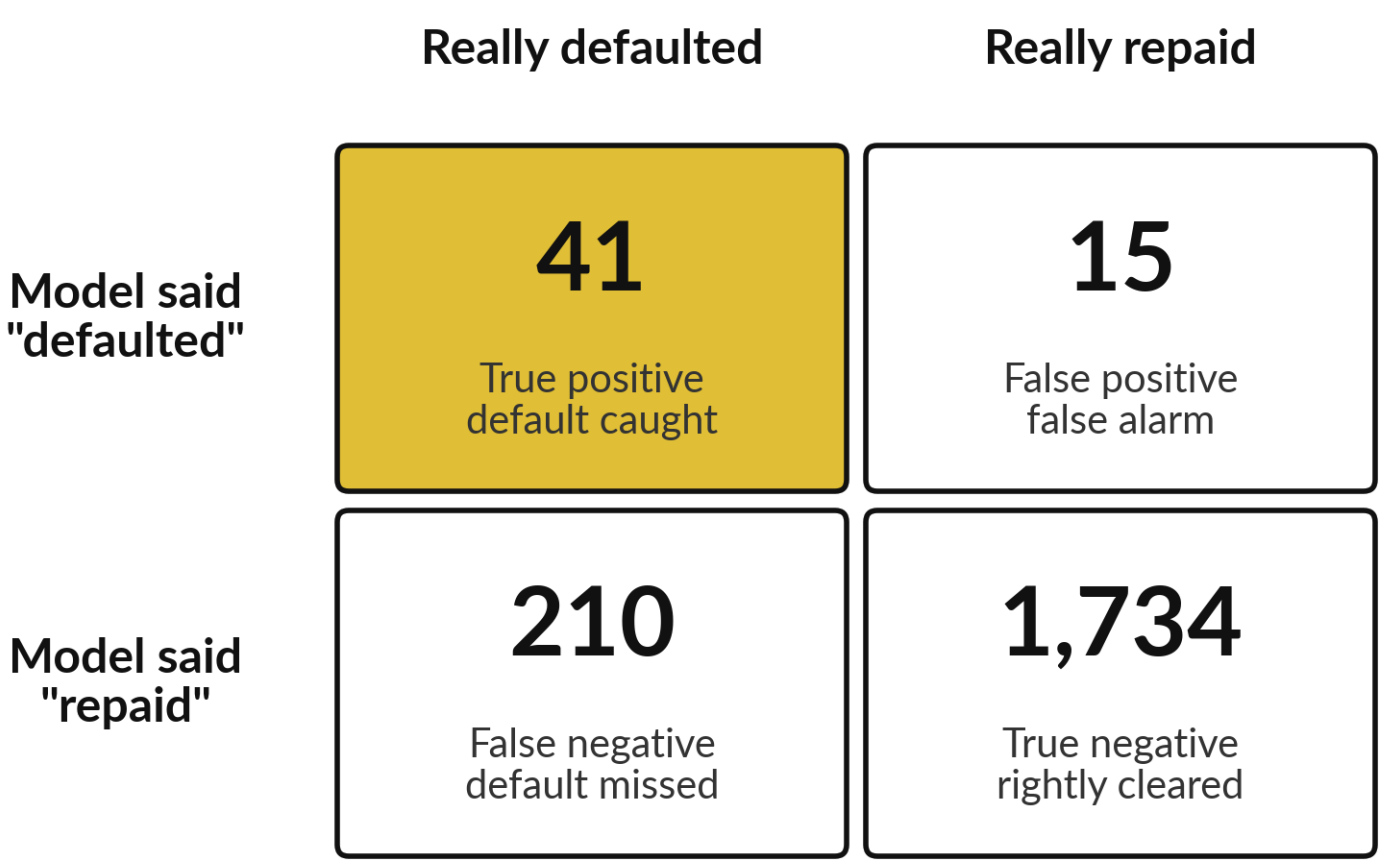
model.fit learns from the training set. model.predict returns a prediction for each test loan.

Test accuracy 0.8875 against 0.8745.

Over ten different splits: 0.882 to 0.893.

The tree asks about instalment_to_income first.

The confusion matrix shows what accuracy hides: 210 defaults missed



The tree catches 41 of the 251 defaults.

Recall 0.16: about 1 default in 6 is caught.

Precision 0.73: of the 56 loans flagged, 41 really defaulted.

Over ten splits, 30 to 55 defaults caught.

Accuracy alone ranks the models wrongly: the Excel rule has lower accuracy and finds far more defaults

	Predictor on the test set	Accuracy	Defaults caught	Recall
	**Baseline model: always repaid**	0.8745	0 of 251	0.00
	**Excel rule: instalment_to_income above 0.6**	0.8005	123 of 251	0.49
	**Decision tree, max_depth=3**	0.8875	41 of 251	0.16

The Excel rule also raises 271 false alarms; the tree raises 15. Which error costs more is a business question. Session 15 turns it into a decision threshold (rule: 0.0909 with illustrative costs).

Where you will use this at work: predict which cases need attention first

	Your job	The question you can answer	The tools you would use
	**Credit analyst at a bank or microfinance lender in Kampala**	Which loan applications are likely to default, using only what is known at approval?	pandas and scikit-learn, with a baseline model first
	**Revenue officer at a district local government**	Illustrative: which taxpayers are likely to file late?	pandas and scikit-learn, results shared in Excel
	**Customer care lead at a mobile money network**	Illustrative: which customers are likely to leave next month?	scikit-learn, Power BI for the dashboard
	**Data officer at a health programme**	Illustrative: which clients are likely to miss a follow-up visit?	scikit-learn, with the test set held back

Three habits that make a model result trustworthy

	Habit	In practice	Why
	**Fix the random seed**	random_state=42 in train_test_split and in the model	A colleague repeats your result exactly (reproducibility).
	**Keep the test set untouched**	Score it once, at the end	Otherwise the final score is too optimistic.
	**Write a data cleaning log**	2 ages replaced, 379 incomes imputed, 1 feature built, 1 column left out	Anyone can audit what you changed.

Read the recipe before the lab: split, baseline, fit, predict

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
baseline_model = DummyClassifier(
    strategy='most_frequent').fit(X_train, y_train)
model = DecisionTreeClassifier(
    max_depth=3, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

In plain words

Lines 1 and 2: split into training and test sets.

Lines 3 and 4: the baseline model learns the majority class.

Lines 5 to 7: make the first model and fit it on the training set.

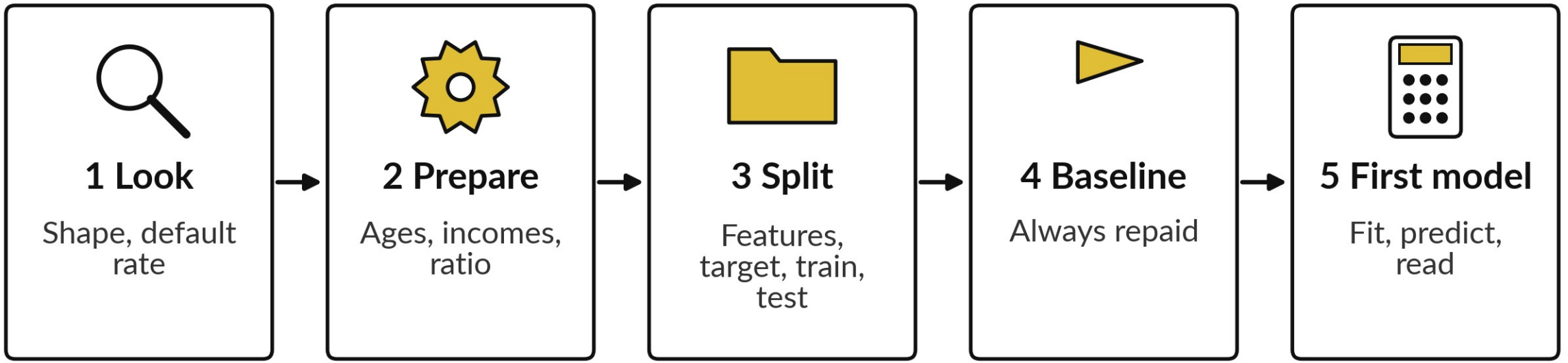
Line 8: predict the test set. y_pred holds one 0 or 1 per test loan.



Part 2: the machine learning lab

Close the slides. Open the notebook.

The lab has five steps and ends with a prepared file and a three-sentence note



Predict, run, read, change in each step. Checks tell you when your numbers match.

Check your numbers before you move on

What you compute	You should get
Shape of the loans, and the default rate	8,000 rows, 18 columns, 0.1255
Impossible ages, and the median of the valid ages	2 and 36
Blank incomes, and the median income	379 and 642,600
Median instalment-to-income ratio, and default rate in its top quarter	0.29 and 27.8 percent
Shape of X	8,000 rows, 12 columns
Test loans, and defaults in the test set	2,000 and 251
Baseline accuracy on the test set	0.8745
Tree accuracy and recall on the test set	About 0.8875 and 0.16

Tree numbers can differ a little between library versions. Ten other splits give accuracy 0.882 to 0.893 and recall 0.12 to 0.22.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**KeyError or AttributeError on a column name**	A column name is misspelt, for example monthly_income instead of monthly_income_ugx.	Print loans.columns and copy the name.
✗	**income_missing is all zeros**	You filled the blanks before making the flag.	Make the flag first. Reload the file and rerun.
✗	**ValueError: Input contains NaN**	A blank is still in X.	Check X.isna().sum() and fill the incomes.
✗	**Very high accuracy of 0.99 or more**	collection_calls is in the features, so the answer leaks.	Remove collection_calls and rerun.
✗	**Tree accuracy is below the baseline**	You scored the training set, or the split used the wrong y.	Score X_test and y_test only.

Your keepsake: a prepared file and a three-sentence note on the first model

	What you keep	What it holds	Where it comes from
	**loans_prepared.csv**	8,000 rows: 12 features and the target default, with no blanks	Notebook step 5
	**Note, sentence 1**	Did the first model beat the baseline model, and by how many points?	Notebook step 5
	**Note, sentence 2**	How many of the real defaults did it catch?	Confusion matrix
	**Note, sentence 3**	What does accuracy hide, and what would you try next?	Slides on the accuracy paradox

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**A model predicts repaid for every loan and scores 0.8745 on the test set. What is it called, and why is the score not impressive?**	A baseline model. The score is just the share of the majority class, and the model catches no defaults.
	**Why do we make the missing-indicator flag before we fill the blanks?**	After filling, the blanks are gone, so the flag could not mark them.
	**Why is collection_calls left out of the features?**	It is recorded after the loan goes bad, so it leaks the answer. This is data leakage.

Save your notebook, your prepared file and your three-sentence note

What you keep from today

Keep your notebook and loans_prepared.csv, a clean file of 12 features and the target, with a three-sentence note: did the first model beat the baseline model, by how much, and what does accuracy hide?

Our question today

What does it mean for a computer to learn from examples, and how do we know it did?

Next session

Session 15: Classification, yes or no questions. We predict default with logistic regression and a decision tree.

Take away

Your lab files

Notebook and prepared file

Your note

Model against baseline

Next session

Classification